

FC申し込み管理アプリ

ファンクラブ公演の申し込み調整・当落管理・公演記録を行うWebアプリ。

技術スタック

役割	技術
DB	Neon (PostgreSQL)
ORM	Drizzle ORM + drizzle-kit
Frontend/API	Next.js (App Router)
Hosting	Vercel
画像解析	Claude Vision API

セットアップ

```
bash

npm install
cp .env.example .env.local
# .env.local に DATABASE_URL と ANTHROPIC_API_KEY を設定
npx drizzle-kit generate
npx drizzle-kit migrate
```

データモデル概要

members	名義マスタ (A/B/Cなど、稀に増減)
productions	ツアー・演目マスタ (申し込みルールを保持)
performances	公演日程 (productionに紐付き)
entries	申し込みエントリー (performance × member)

重要な設計ポイント

一本釣りフラグはDBカラムではなくViewで導出 (`entries_with_ikkonzuri`) ビューを参照することで、「同一ツアー×同一名義のエントリが1件のみ」を動的に判定。エントリの追加・削除に追従して自動更新される。

同行者タイプの3パターン

- `fc_member`: companion_member_id に名義IDを設定
- `general_email`: companion_email にメアドを設定
- `none`: 同行者なし (一本釣りの基本形)

本人確認レベルはproductionに持つ `id_verification = "face_auth"` の場合、
`can_pass_id_verification = false` の名義（名義Cなど）はアプリ側でエントリ作成時にバリデーションエラーとする。

申込時同行者必須の場合の上限チェック `companion_timing = "at_entry"` の場合、同一ツアーで同一名義を同行者に指定できるのは1公演のみ。これもアプリ側のバリデーションで制御。

画面構成（予定）

/	ダッシュボード（直近の申し込み状況）
/productions	ツアー・演目一覧
/productions/new	ツアー・演目登録（スクショアップロード → 日程自動読み取り）
/productions/[id]	公演日程一覧 + エントリ状況
/entries/[id]	エントリ詳細（当落・入金・座席の更新）
/analysis	当落分析
/archive/new	過去半券の取り込み（画像 → 座席情報）

スクショ取り込みフロー

1. `/productions/new` でスクショをアップロード
2. Claude Vision API が日程を構造化JSON化
3. 確認画面でプレビュー → 修正があれば編集
4. 保存でproductionとperformancesを一括作成
5. 各performanceに対してentriesを手動で作成（申し込み名義を選択）

環境変数

```
DATABASE_URL=postgresql://...@...neon.tech/fc_app
ANTHROPIC_API_KEY=sk-ant-...
```